

# Lab 07: Multi-Agent Signaling Game

## Objective

Build a 2-agent signaling game, run it with learning, and measure emergent communication via mutual information.

## Prerequisites

- Completed Labs 01–06
- Understanding of mutual information and multi-agent dynamics

## Part 1: Setting Up the Agents

**Goal:** Create sender and receiver agents for a 2-state signaling game.

1. Define the sender's model: A = identity (observes true state), B = identity, C = [reward preference], D = uniform.
2. Define the receiver's model: A = identity (maps signal to observation), B shaped by the sender's action, C = same reward preference.
3. Instantiate both `ActiveInferenceAgent` instances with  $\gamma = 4.0$ .

```
import numpy as np
from active_inference.agent import GenerativeModel, ActiveInferenceAgent

# TODO: Define sender and receiver models
# TODO: Create agents
```

**Response:**

*Write your response here*

## Part 2: Running the Signaling Loop

**Goal:** Run 100 rounds of the signaling game.

1. At each round: randomly sample a world state (0 or 1).
2. Sender observes the state and selects a signal.
3. Receiver observes the signal and selects a direction.
4. Record success (receiver chose correct direction) and the (signal, state) pair.

```
signals = []
states = []
successes = []

for t in range(100):
    world_state = np.random.choice(2)
    signal = agent_sender.step(world_state)
    direction = agent_receiver.step(signal)
    success = (direction == world_state)

    signals.append(signal)
    states.append(world_state)
    successes.append(success)
```

**Response:**

*Write your response here*

## Part 3: Adding Learning

**Goal:** Add Dirichlet learning so agents can develop a communication protocol.

1. Add `update_dirichlet_A()` for both agents after each round.
2. Update expected A-matrices for subsequent inference.
3. Run 200 rounds with learning enabled.
4. Compare success rate in rounds 1–50 vs rounds 151–200.

**Response:**

*Write your response here*

## Part 4: Measuring Mutual Information

**Goal:** Track mutual information between signals and world states over time.

1. Compute MI using a sliding window of 20 rounds.
2. Build the joint distribution from the windowed (signal, state) pairs.
3. Plot MI over time. It should increase if communication emerges.

```
from active_inference.math import mutual_information

mi_history = []
window = 20
for t in range(window, len(signals)):
    joint = np.zeros((2, 2))
    for s, sig in zip(states[t-window:t], signals[t-window:t]):
        joint[sig, s] += 1
    joint /= joint.sum()
    mi_history.append(mutual_information(joint))
```

**Response:**

*Write your response here*

## Part 5: Analysis Questions

1. Did mutual information increase over time? What was the final MI compared to the maximum ( $\ln 2 \approx 0.693$ )?
2. Did the sender develop a consistent mapping between world states and signals? What was the mapping?
3. How many rounds were needed before the receiver's accuracy exceeded 80%?

**Response:**

*Write your response here*

## Summary

| Skill                                          | Library Component                                        | Status |
|------------------------------------------------|----------------------------------------------------------|--------|
| Build multi-agent Active Inference systems     | Multiple <code>ActiveInferenceAgent</code> instances     |        |
| Implement a signaling game loop                | Sender-receiver architecture                             |        |
| Add Dirichlet learning to multi-agent settings | <code>update_dirichlet_A()</code> per agent              |        |
| Measure emergent communication                 | <code>mutual_information()</code> on joint distributions |        |
| Analyze communication development over time    | Sliding-window MI tracking                               |        |